Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

TRiM - A Multi Tenant SaaS Application

By
Ariel C. Nunes

March 30, 2026

B.Sc. (Hons) in Software Development

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.1 Project Repository . . . 2
1.2 Context . . . 2
1.3 Project Objectives . . . 3
1.4 Dissertation Focus . . . 4
1.5 Success Metrics . . . 4
1.6 Chapters Overview . . . 5

2 Methodology **7**
2.1 Development Approach . . . 7
2.1.1 Requirements Gathering . . . 10
2.2 Generative AI Usage . . . 10
2.2.1 Code Generation with GitHub Copilot . . . 10
2.2.2 Testing Automation with Claude Code . . . 10
2.2.3 Code Review . . . 10
2.3 Research Approach . . . 10
2.3.1 Literature Search . . . 10
2.3.2 Choosing the Technology Stack . . . 10
2.4 Evaluation Methodology . . . 11
2.4.1 Benchmarking With Gaitling Load Testing . . . 11
2.4.2 Test Data Generation With java-seeder . . . 11
2.4.3 Performance Metrics . . . 11

3 Technology Review **12**
3.1 Multi-Tenant Database Architectures . . . 12
3.1.1 Different Types of Multi-Tenant Database Designs . . . 12
3.1.2 Trade-offs: Isolation vs. Efficiency vs. Cost . . . 13
3.1.3 Data Isolation Techniques . . . 14
3.2 Related Works . . . 15
3.2.1 Proposed Multi-Tenant SaaS Database Design Patterns . . . 15
3.2.2 Literature Reviews . . . 15

4 System Design **17**
4.1 System Overview . . . 18
4.1.1 Architecture Diagram . . . 18
4.1.2 RESTful API Design . . . 18
4.1.3 Technology Stack . . . 18
4.2 Multi-Tenant Database Design . . . 18
4.2.1 Entity-Relationship Model . . . 18
4.2.2 Migrating From Single to Multi-Tenant . . . 18
4.2.3 Tenant-Aware Schema . . . 18
4.2.4 Row Level Security vs Application-Level Isolation . . . 18
4.3 Multi-Tenant Implementation . . . 18
4.3.1 TenantFilter & TenantContext . . . 18
4.3.2 Subdomain-based Routing . . . 18
4.3.3 Cross-Subdomain Token Exchange . . . 18
4.4 Business Operations . . . 18
4.4.1 Booking & Payment Flow . . . 18
4.4.2 Race Condition Prevention When Booking . . . 18
4.4.3 Authentication & Security . . . 18
4.4.4 JWT & Role-Based Access Control . . . 18
4.5 Frontend Architecture . . . 18
4.5.1 Component Structure . . . 18
4.5.2 State Management . . . 18
4.5.3 Responsive Design . . . 18
4.5.4 Integration with Backend API . . . 18
4.6 Deployment Architecture . . . 18
4.6.1 Docker . . . 18
4.6.2 Deploying Application to a VPS . . . 18

5 System Evaluation **20**
5.1 Performance Evaluation . . . 20
5.1.1 Database Gaitling Benchmarking - Load Testing . . . 20
5.2 Security Evaluation . . . 20
5.2.1 Tenant Isolation Verification . . . 20
5.3 Critical Analysis . . . 20
5.3.1 Strengths . . . 20
5.3.2 Limitations . . . 20

6 Conclusion **21**
6.1 Future Work . . . 21

List of Figures

2.1 GitHub Projects Kanban board used to track development progress. 8
2.2 Example user story with acceptance criteria, priority, and technical notes. 9
2.3 Benchmarking API in IntelliJ IDE. 11

3.1 Multi-tenant database isolation patterns 13

List of Tables

Chapter 1

Introduction

1.1 Project Repository

The source code for this project is available at `https://github.com/ArielCNunes/TRiM-FYP`. The repository is organised into three top-level directories:

- **`backend/`**: Contains the Spring Boot REST API, structured as a standard Maven project. Key packages include `controller`, `service`, `repository`, `entity`, and `tenant`, reflecting the layered architecture and multi-tenant logic at the core of the system. Supporting packages handle payment processing, email notifications, booking reminders, and test data seeding.
- **`frontend/`**: A React application built with TypeScript and Vite, styled with Tailwind CSS. The source is organised into `pages`, `components`, `features`, `hooks`, and `store` directories, separating UI concerns from state management and API integration.
- **`docs/`**: Contains this dissertation in LaTeX format, along with records of client and supervisor meetings maintained throughout the project.

1.2 Context

Small businesses such as barber shops that rely solely on walk-in customers face several operational challenges: limited business insights, customer friction when services are unavailable at their preferred time, and the burden of manual bookkeeping. Digitalising these operations bridges the gap between the business and its customers, leading to a more established market presence and streamlined day-to-day management.

TRiM is a platform designed to support the full range of barber shop operations, from customer bookings and payment processing to employee management and business analytics. Rather than serving a single business, *TRiM* is built to allow multiple businesses to operate on the platform independently, each with its own isolated data and user base. One instance of the software serves multiple businesses, this is known as multi-tenant architecture, and it is the core focus of this dissertation.

1.3 Project Objectives

The objective of this project is to deliver a real and robust software solution to local service-based businesses. The following goals have been defined:

- **Customer Booking System**: Enable customers to book services online, select their preferred barber, choose from categorized services, and pick available time slots while the system prevents double bookings and scheduling conflicts.
- **Payment Integration**: Integrate Stripe for secure online payment processing, supporting both full online payments and deposit-based bookings where customers pay a percentage upfront and settle the outstanding balance in-shop, driven by real client requirements.
- **Business Administration**: Provide business owners with a comprehensive admin dashboard to manage service categories, individual services with pricing and duration, and employee (barber) profiles. Include analytics such as total revenue, booking statistics, and popular services.
- **Barber Management**: Allow barbers to manage their own availability by setting weekly working hours, defining break periods, and viewing/managing their upcoming bookings with the ability to confirm, complete, or handle cancellations and no-shows.
- **Customer Management**: Enable administrators to view customer history, track no-show counts, and blacklist problematic customers to protect business operations.
- **Multi-Tenant Architecture**: Design the platform to support multiple independent businesses, each with their own subdomain (business slug), isolated data, and separate admin/barber/customer user bases.

- **Role-Based Access Control**: Implement distinct user roles (CUSTOMER, BARBER, ADMIN) with appropriate permissions and tailored interfaces for each role's specific needs.

1.4 Dissertation Focus

This dissertation focuses primarily on the architectural decisions behind *TRiM*, particularly the database design patterns and data isolation techniques employed. Through *TRiM*, we explore how to build a multi-tenant software application that is both scalable and robust, leveraging efficient design patterns that allow the system to grow without significant cost or performance overhead. We also investigate how multiple businesses can share the same platform whilst maintaining strong performance guarantees and strict security boundaries.

1.5 Success Metrics

From a functional standpoint, the system must handle all core operations reliably: bookings should complete without conflicts, payments should process securely, and the admin dashboard should provide accurate insights into business operations.

Beyond functionality, performance represents a critical area of evaluation. The system will be benchmarked under various conditions to assess its resilience. This includes measuring response times for key API endpoints, evaluating database performance as the number of tenants and bookings increases, and verifying that heavy usage by one business does not degrade the experience for others. Tenant isolation extends beyond security considerations, it also ensures fair distribution of system resources across all users.

Security constitutes another essential metric. With multiple businesses and their customer data at stake, the system must demonstrate robust protection mechanisms. Cross-tenant data leakage would represent a critical failure, necessitating thorough testing of all isolation mechanisms to prevent such occurrences.

Finally, usability remains a key consideration. The primary objective of *TRiM* is to streamline operations for business owners and their customers. An unintuitive interface or cumbersome booking flow would undermine the value of the underlying architecture.

1.6 Chapters Overview

Chapter 2: Methodology

Outlines the development approach used in the project. It describes the agile workflow with a Kanban board in GitHub Projects, the user stories used to gather requirements, and the feedback loop with weekly supervisor meetings and regular client demonstrations with V7 Barbers. The chapter also covers the tools used for development, the role of generative AI, testing, and the evaluation strategy built around Gatling load testing and seeded test data.

Chapter 3: Technology Review

Looks at the three primary multi-tenant database architectures: separate databases, separate schemas, and shared schemas. Analyses their trade-offs in terms of isolation, cost, and scalability. The chapter surveys data isolation techniques including filter-based and permission-based approaches such as PostgreSQL Row-Level Security, and reviews related work on adaptive schema design, and recent benchmarking frameworks. It identifies the gap that motivates this project: a lack of empirical evidence on combining shared-schema designs with database-level security enforcement.

Chapter 4: System Design

Presents the architecture of *TRiM*, from the RESTful API and entity-relationship model to the multi-tenant implementation built on subdomain-based routing, tenant-aware schema design, and Row-Level Security. It details core operational flows including booking and payment processing with Stripe, race condition prevention, JWT-based authentication with role-based access control, and cross-subdomain token exchange. The chapter also covers the React frontend architecture and the Docker-based deployment strategy.

Chapter 5: System Evaluation

Evaluates the system against the success metrics defined in this chapter. Performance is assessed through Gatling load tests that measure API response times and database behaviour as tenant and booking volumes increase. Security is verified through tenant isolation testing to confirm that no cross-tenant data leakage occurs. The chapter concludes with a critical analysis of the system's strengths and the limitations of the current design.

Chapter 6: Conclusion

Reflects on the extent to which the project objectives were met, sums up the key findings from the system evaluation, and discusses the broader implications of the architectural choices made. It concludes with directions for future work, including potential improvements to the platform and areas where the multi-tenant design could be extended.

Chapter 2

Methodology

2.1 Development Approach

Agile Development Techniques Applied

The development of *TRiM* followed an agile methodology (where possible, given the scope of the project), incorporating several techniques to ensure continuous progress, stakeholder alignment, and structured delivery of features throughout the project lifecycle.

A Kanban board was maintained using GitHub Projects to track the progress of all development work. Each item on the board represented a unit of work that came from a user story, and moved through columns such as *Backlog*, *In Progress*, and *Done* as development progressed, see Figure 2.1. This provided a clear visual overview of the project and helped prioritise work based on immediate needs and client feedback.

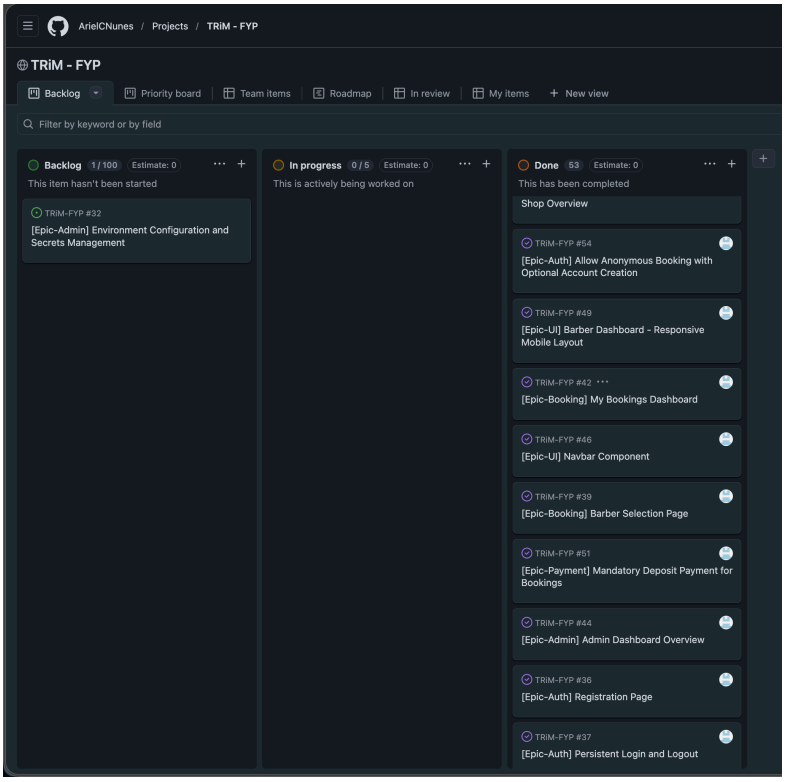

Figure 2.1: GitHub Projects Kanban board used to track development progress.

Requirements were captured as user stories, each documented as a GitHub Issue following a consistent structure. Every user story included four sections: a **User Story** statement written in the standard "As a [role], I want [feature], so that [benefit]" format; a set of **Acceptance Criteria** expressed using Given/When/Then conditions that defined the expected behaviour; a **Priority** classification (Must Have, Should Have, Could Have); and **Technical Notes** outlining relevant implementation details such as API endpoints, and UI behaviour. Figure 2.2 illustrates a user story from the project. This format ensured that each piece of work had clearly defined scope and testable outcomes before development began.

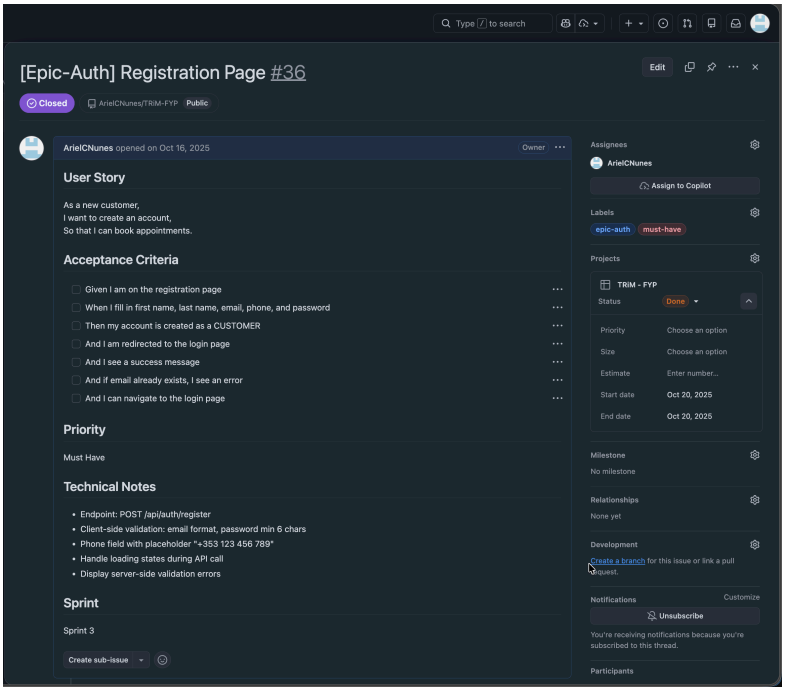

Figure 2.2: Example user story with acceptance criteria, priority, and technical notes.

Regular communication was maintained throughout the project. Weekly meetings were held with the project supervisor to review progress, discuss technical decisions, and receive academic guidance. These meetings ensured that the project remained aligned with the FYP objectives and that research and implementation efforts were progressing. In parallel, regular meetings were conducted with the client, Jean, the owner of V7 Barbers. These meetings served as the primary mechanism for requirements gathering and validation. Early sessions focused on reviewing initial user stories and identifying additional requirements such as deposit-based payments and customer blacklisting, while later meetings involved live demonstrations of the system to get feedback on usability and functionality. This continuous feedback loop between developer, supervisor, and client ensured that *TRiM* was shaped by real business needs rather than assumptions. All meetings were documented for reference.

Version Control and Development Tools

All source code was managed using Git and hosted on GitHub. Commits were made frequently to maintain a clear history of changes throughout the project. Development was carried out primarily using IntelliJ IDEA for the Spring Boot

backend and Visual Studio Code for the React frontend. GitHub Codespaces was also used on occasion for remote development when working from a different machine. Docker Compose was used locally to run the PostgreSQL database alongside the application during development.

2.1.1 Requirements Gathering

- Real client - User stories

2.2 Generative AI Usage

2.2.1 Code Generation with GitHub Copilot

2.2.2 Testing Automation with Claude Code

2.2.3 Code Review

2.3 Research Approach

2.3.1 Literature Search

- Search strings and databases used - Inclusion and exclusion criteria

2.3.2 Choosing the Technology Stack

- Reasons for selecting Java, Spring Boot, React, PostgreSQL, etc.

2.4 Evaluation Methodology

2.4.1 Benchmarking With Gaitling Load Testing

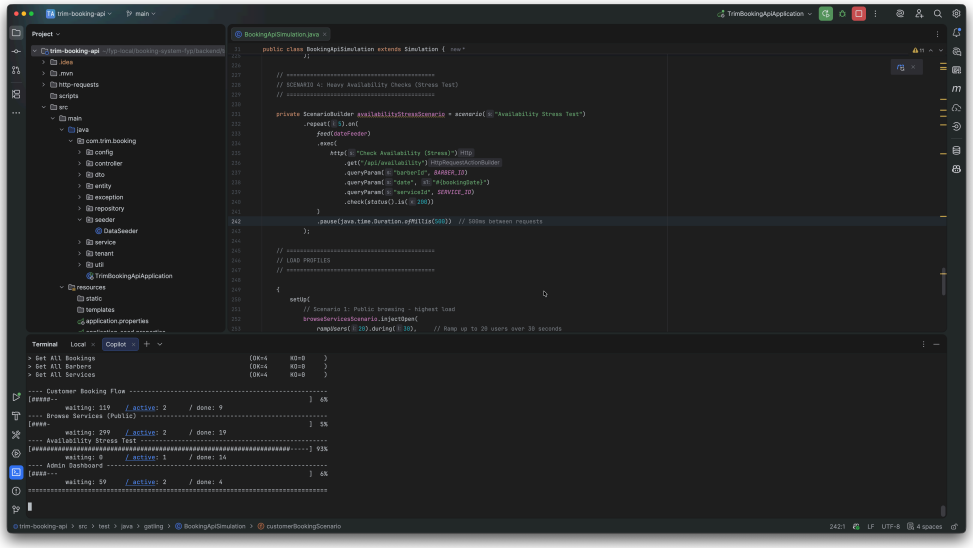

Figure 2.3: Benchmarking API in IntelliJ IDE.

2.4.2 Test Data Generation With java-seeder

2.4.3 Performance Metrics

Chapter 3

Technology Review

3.1 Multi-Tenant Database Architectures

A multi-tenant system provides one shared instance of an application and its infrastructure to multiple users, referred to as tenants, while maintaining the logical separation of their data. In a SaaS context, tenants are typically organisations or businesses that each require their own private view of the data. How that separation is achieved at the database level is one of the most important design decisions in a SaaS application, as it determines the system's performance, security, operational cost, and ability to scale [1].

3.1.1 Different Types of Multi-Tenant Database Designs

The multi-tenant database design can be broken down into three primary isolation models, as defined by Karatąs et al. [2] and illustrated in Figure 3.1.

Separate Application, Separate Database gives each tenant a fully independent application instance and database. Data isolation is guaranteed: there is no shared infrastructure at the database layer, so there is no risk of one tenant accessing another's data. The trade-off is cost and operational overhead. Every new tenant requires provisioning an additional stack, and hardware resources cannot be shared across tenants, making this model expensive to operate at scale [2].

Shared Application, Separate Database retains a single application instance but still provides each tenant with its own database. This reduces application-layer overhead while preserving data isolation. It's still expensive in terms of storage and database management, since the number of databases grows linearly with the number of tenants.

Shared Application, Shared Database is the most resource-efficient model. All tenants share the same application and the same database, with data partitioned logically rather than physically. This model has two variants. In *Shared*

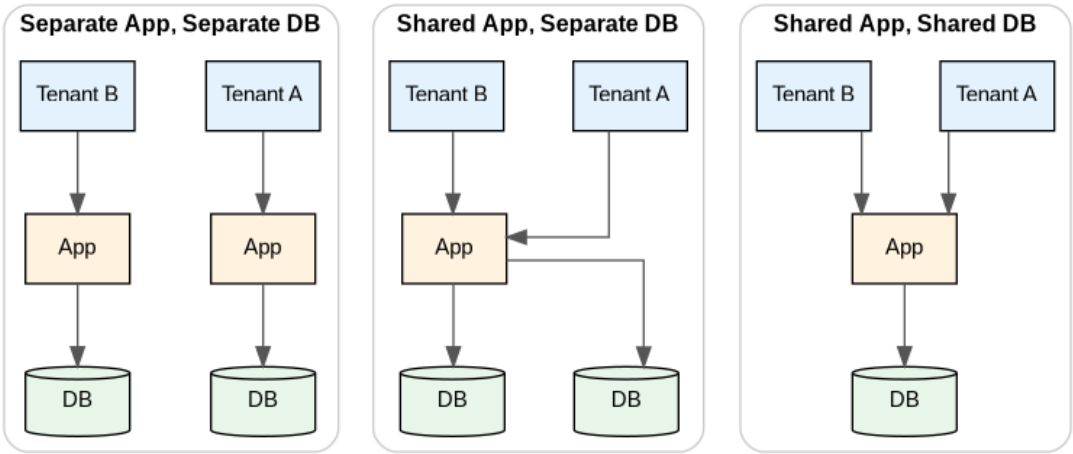

Figure 3.1: Multi-tenant database isolation patterns

Database, Separate Schema, each tenant receives its own set of tables within the shared database. In *Shared Database, Shared Schema*, all tenants share the same tables, and their rows are distinguished by a `tenant_id` column. The shared schema variant offers the highest resource utilisation and the lowest per-tenant cost, but it places a burden on the data isolation mechanisms, since a missing or incorrect filter predicate could expose one tenant's data to another [1].

3.1.2 Trade-offs: Isolation vs. Efficiency vs. Cost

The selection of an isolation model is not a decision that can optimise all dimensions simultaneously. Wang et al. [1] established the foundational tension: higher isolation means better security but limits the number of tenants a system can support on fixed hardware. Zhou et al. [3] quantify this directly on IBM DB2, demonstrating that a shared table model (S3) supports approximately 400 concurrent tenants on the same hardware that a separate database model (S1) can sustain only around 8. The capacity advantage of the shared approach is significant, but it depends on effective isolation being built into the system.

Cost is yet another concern. Wang et al. [1] categorise multi-tenant system costs across three dimensions: infrastructure (hardware, software, and utilisation), management (lifecycle, monitoring, and backup), and development (tenant-specific customisation). Shared architectures reduce infrastructure and management costs substantially. Aulbach et al. [4] and Yaish and Goyal [5] both demonstrate that shared schemas lower Total Cost of Ownership by eliminating per-tenant schema provisioning and by enabling self-service configuration. However, neither work

quantifies how the cost of securing a shared schema compares to the cost of stronger physical isolation.

Performance also needs to be taken into account. Wang et al. [1] measured that adding Label-Based Access Control (LBAC) - a database-level security mechanism - reduced throughput by 15% compared to less secure approaches. Ni et al. [6] propose the ADAPT methodology, which improves query performance by 2–3× over static shared-schema approaches by dynamically clustering attributes based on workload, but their evaluation explicitly assumes application-level filtering and tests no security mechanisms. The pattern is consistent across the literature: studies that optimise performance assume the security problem is solved elsewhere, and studies that address security do not measure the performance impact of their mechanisms [2, 7].

3.1.3 Data Isolation Techniques

Within the shared database, shared schema model, the mechanisms used to enforce tenant data isolation fall into two categories: filter-based and permission-based.

Filter-based isolation is implemented at the application layer by appending a tenant identifier predicate to every query, typically expressed as `WHERE tenant_id = ?`. Wang et al. [1] identify this as the most common approach in practice due to its simplicity. Its critical weakness is that isolation is only as reliable as the application code: a missing predicate, a buggy query, or a SQL injection vulnerability can expose data across tenant boundaries. Gupta, Kumar, and Agnihotri [8] extend this approach with session-based identifiers that rotate periodically, reducing injection risk, but their mechanism remains dependent on the application being free of errors.

Permission-based isolation enforces tenant boundaries at the database engine level, independently of the application. PostgreSQL Row-Level Security (which has been used in this project) is a prominent example: policies are attached directly to tables, and the database engine automatically appends the isolation predicate to every qualifying query before execution, regardless of what the application sends. Factor et al. [9] formalise the principles underpinning this approach in their SLIM (Secure Logical Isolation for Multi-tenancy) model, based on least privilege, tenant containment, escalation avoidance, and minimising attack surface. The advantage of database-level isolation is that it acts as a safety net: even if the application layer has a bug, the database will not return rows belonging to another tenant. The documented trade-off is a performance overhead from the additional policy evaluation on every query.

3.2 Related Works

3.2.1 Proposed Multi-Tenant SaaS Database Design Patterns

Several studies have proposed specific techniques for managing multi-tenant data that are directly relevant to the design decisions made in this project.

Ni et al. [6] introduce **ADAPT** (Adaptive Database Schema Design for Multi-Tenant Data Management), a methodology that dynamically reorganises table schemas by clustering attributes based on observed query workloads. ADAPT achieves a 2–3× throughput improvement over static shared-schema designs and demonstrates reasonable scalability up to 4,000 tenants, at which point the alternative individual-table approach degrades severely. Its limitation for this project's context is that it assumes application-level filtering and does not evaluate how it would work with database-level security policies.

Mangwani and Tokekar [10] evaluate microservice architectures as a scalability mechanism for multi-tenant SaaS systems. Their findings show that at low concurrency (around 50 users), monolithic and microservice architectures perform similarly, but at high load (300+ users), microservices offer lower response times, higher throughput, and approximately 11% better CPU utilisation. This supports the case for modular application design but treats the database as a black box, with no evaluation of how schema design choices interact with service decomposition.

Sharma and Kaur [11] demonstrate MongoDB sharding as an alternative to relational multi-tenant designs, enabling horizontal scaling through data partitioning across nodes with automatic load balancing. While this illustrates the growing relevance of NoSQL approaches, the study does not compare its performance or isolation guarantees against optimised relational alternatives, leaving the trade-off between schema flexibility and data isolation unknown.

Most recently, Yin et al. [12] propose the MTD-DS benchmark, arguing that existing benchmarks such as TPC-H and TPC-C fail to capture SLA-relevant metrics like query completion time per tenant and tenant prioritisation. Their work introduces SLA-aware pricing models including a Query-Level SLA-Aware (QLSA) model that ties provider cost to actual service level objective satisfaction. While this framework represents a conceptual advance, it remains a benchmark model with no empirical validation against real-world systems.

3.2.2 Literature Reviews

Two prior systematic reviews are most directly relevant to this work and establish the context into which this project's design decisions fit.

Karataş et al. [2] conducted a broad systematic mapping study covering 407 papers from 2000 to 2017. Their key quantitative finding is that 65% of papers preferred a shared database strategy, and 58% were ideological proposals without implementation. The study is descriptive rather than evaluative: it reports which strategies are preferred but does not analyse why or evaluate combined approaches. Its 2017 cutoff also means it predates significant developments in containerised deployment and modern database security tooling.

Battula [7] provides a more recent (2024) systematic review focused specifically on security mechanisms in multi-tenant cloud databases, analysing 25 papers from 2012 to 2024. It comprehensively covers encryption, access control, and threat detection, but treats performance and scalability as secondary concerns. Like Karataş et al., it evaluates mechanisms in isolation and identifies no integrated approaches combining database-level security with performance optimisation techniques.

The gap identified by both reviews, and confirmed by the broader set of papers analysed, is the absence of empirical evaluation of combined approaches. No study evaluates how a database-level security mechanism such as RLS performs when applied alongside a shared schema optimised for multi-tenant query patterns, nor how much of the performance benefit claimed by approaches like ADAPT survives the addition of policy-based isolation. Zhu et al. [13] highlight an additional dimension of this problem: optimising database performance for one tenant can degrade it for others, a conflict that existing research leaves unresolved.

This project addresses a subset of this integration gap directly. TRiM adopts a Shared Application, Shared Database, Shared Schema design, the pattern associated with the highest resource efficiency, and enforces tenant isolation using PostgreSQL Row-Level Security rather than application-layer filtering alone. The load testing results presented in Chapter 5 provide empirical evidence on the performance overhead of this combination under real data volumes, contributing practical findings to the field.

Chapter 4

System Design

4.1 System Overview

4.1.1 Architecture Diagram

4.1.2 RESTful API Design

4.1.3 Technology Stack

4.2 Multi-Tenant Database Design

4.2.1 Entity-Relationship Model

4.2.2 Migrating From Single to Multi-Tenant

4.2.3 Tenant-Aware Schema

4.2.4 Row Level Security vs Application-Level Isolation

4.3 Multi-Tenant Implementation

4.3.1 TenantFilter & TenantContext

4.3.2 Subdomain-based Routing

4.3.3 Cross-Subdomain Token Exchange

4.4 Business Operations

4.4.1 Booking & Payment Flow

4.4.2 Race Condition Prevention When Booking

4.4.3 Authentication & Security

4.4.4 JWT & Role-Based Access Control

4.5 Frontend Architecture

4.5.1 Component Structure

and algorithms to illustrate your design.

Chapter 5

System Evaluation

5.1 Performance Evaluation

5.1.1 Database Gaitling Benchmarking - Load Testing

5.2 Security Evaluation

5.2.1 Tenant Isolation Verification

5.3 Critical Analysis

5.3.1 Strengths

5.3.2 Limitations

Chapter 6

Conclusion

6.1 Future Work

Briefly summarise your context and objectives. Remind the reader about the overall rationale and goals of the project. Highlight your findings from the System Evaluation chapter.

Bibliography

[1] Zhi Hu Wang, Chang Jie Guo, Bo Gao, Wei Sun, Zhen Zhang, and Wen Hao An. A Study and Performance Evaluation of the Multi-Tenant Data Tier Design Patterns for Service Oriented Computing. In *2008 IEEE International Conference on E-Business Engineering*, pages 94–101.

[2] Gözde Karata, Ferit Can, Gamze Doan, Cemile Konca, and Akhan Akbulut. Multi-tenant architectures in the cloud: A systematic mapping study. In *2017 International Artificial Intelligence and Data Processing Symposium (IDAP)*, pages 1–4.

[3] Ying Hua Zhou, Qi Rong Wang, Zhi Hu Wang, and Ning Wang. DB2MMT: A Massive Multi-tenant Database Platform for Cloud Computing. In *2011 IEEE 8th International Conference on E-Business Engineering*, pages 335–340.

[4] Stefan Aulbach, Michael Seibold, Dean Jacobs, and Alfons Kemper. Extensibility and Data Sharing in evolving multi-tenant databases. In *2011 IEEE 27th International Conference on Data Engineering*, pages 99–110.

[5] Haitham Yaish and Madhu Goyal. A Multi-tenant Database Architecture Design for Software Applications. In *2013 IEEE 16th International Conference on Computational Science and Engineering*, pages 933–940.

[6] Jiacai Ni, Guoliang Li, Lijun Wang, Jianhua Feng, Jun Zhang, and Lei Li. Adaptive Database Schema Design for Multi-Tenant Data Management. 26(9):2079–2093.

[7] Moshe Battula. A Systematic Review on a Multi-tenant Database Management System in Cloud Computing. In *2024 International Conference on Cognitive Robotics and Intelligent Systems (ICC - ROBINS)*, pages 890–897.

[8] Keshav Gupta, Sandeep Kumar, and Ojaswi Agnihotri. Data isolation in multi-tenant SaaS environment. In *2016 International Conference on Computing, Communication and Automation (ICCCA)*, pages 1290–1292.

[9] Michael Factor, David Hadas, Aner Harnama, Nadav Har'el, Elliot K. Kolodner, Anil Kurmus, Alexandra Shulman-Peleg, and Alessandro Sorniotti. Secure Logical Isolation for Multi-tenancy in cloud storage. In *2013 IEEE 29th Symposium on Mass Storage Systems and Technologies (MSST)*, pages 1–5.

[10] Poonam Mangwani and Vrinda Tokekar. Container Based Scalability and Performance Analysis of Multitenant SaaS Applications. In *2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6.

[11] Aditi Sharma and Parmeet Kaur. Implementing a Multitenant System using a Document-based NoSQL Database. In *2021 5th International Conference on Information Systems and Computer Networks (ISCON)*, pages 1–5.

[12] Shaoyi Yin, Franck Morvan, Jorge Martinez-Gil, and Abdelkader Hameurlain. MTD-DS: An SLA-Aware Decision Support Benchmark for Multi-Tenant Parallel DBMSs. 37(5):2743–2755.

[13] Zekai Zhu, Peiquan Jin, Guorui Huang, Xiaoliang Wang, and Yigui Yuan. MTRP: A High-Performance Cost-Efficient Buffering Scheme for Multi-Tenant Cloud Services. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, pages 1677–1682.